Patient ETL Pipeline Using AWS Glue and Redshift

1. Introduction

In this project, we designed and implemented an end-to-end ETL (Extract, Transform, Load) pipeline using AWS services. The objective was to extract raw data from Amazon S3, transform it using AWS Glue, and load the processed data into Amazon Redshift for analytics.

This pipeline demonstrates how serverless data engineering can be used to process and analyze structured data efficiently.

2. Objectives

- Extract data from multiple CSV files stored in Amazon S3
- Clean and transform the data using AWS Glue (PySpark)
- Store transformed data in Parquet format in S3
- Load the processed data into Amazon Redshift
- Perform analytical queries on the final dataset

3. AWS Services Used

- Amazon S3 – Data storage
- AWS Glue – ETL processing
- AWS Glue Data Catalog – Metadata management
- Amazon Redshift – Data warehouse
- IAM – Access control

4. Dataset Description

4.1 User Data

Contains user details such as:

- UserID
- FirstName
- LastName

- Email
- SignupDate

4.2 Product Data

Contains product information:

- ProductID
- ProductName
- Category
- Price
- StockQuantity

4.3 Order Data

Contains transaction details:

- OrderID
- UserID
- OrderDate
- Amount
- Status

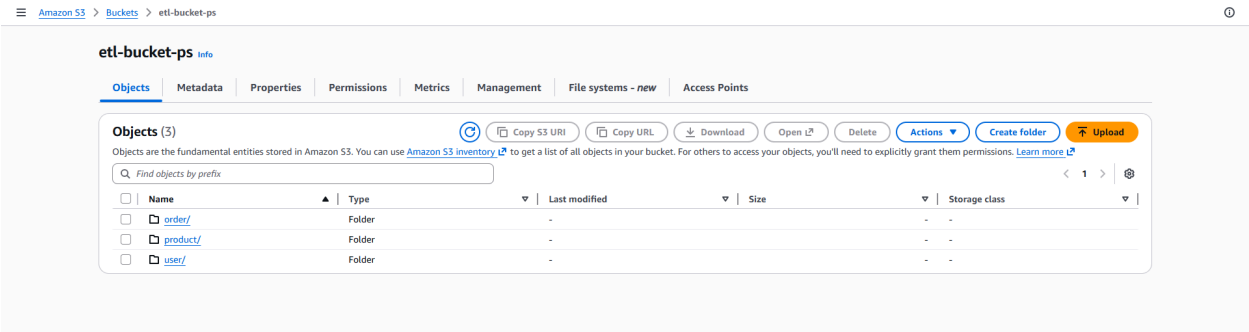
5. Architecture Overview

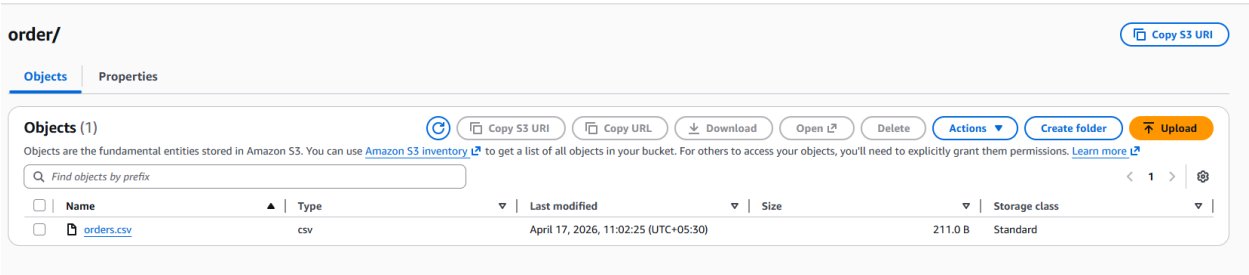
S3 (Raw CSV Data)
↓
Glue Crawler
↓
Glue Data Catalog
↓
Glue ETL Job (PySpark)
↓
S3 (Processed Parquet Data)
↓
Redshift
↓
SQL

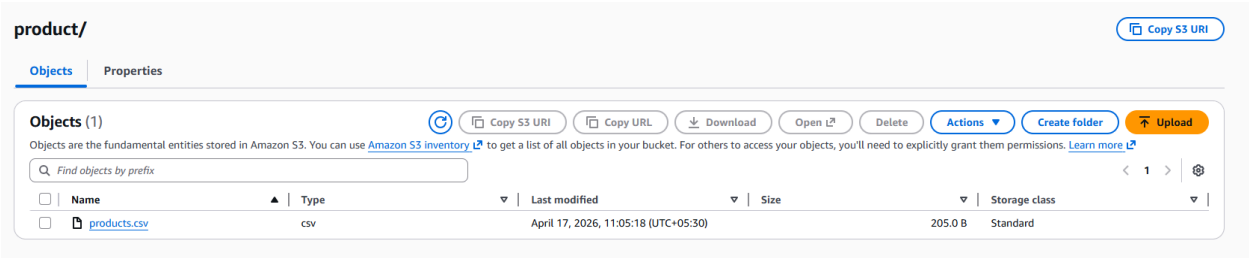
6. Implementation Steps

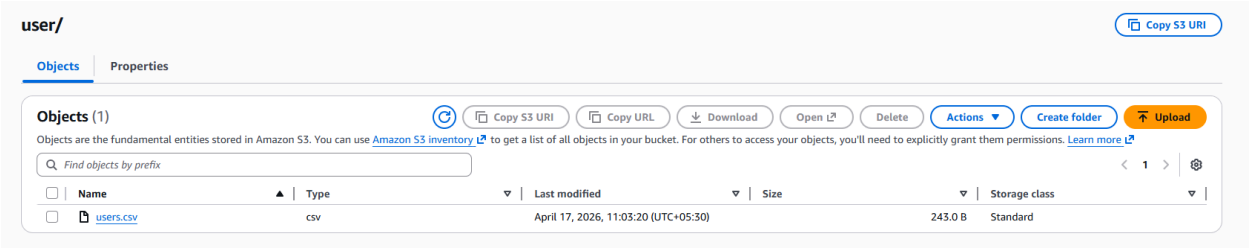
6.1 Data Extraction

- Uploaded CSV files into Amazon S3

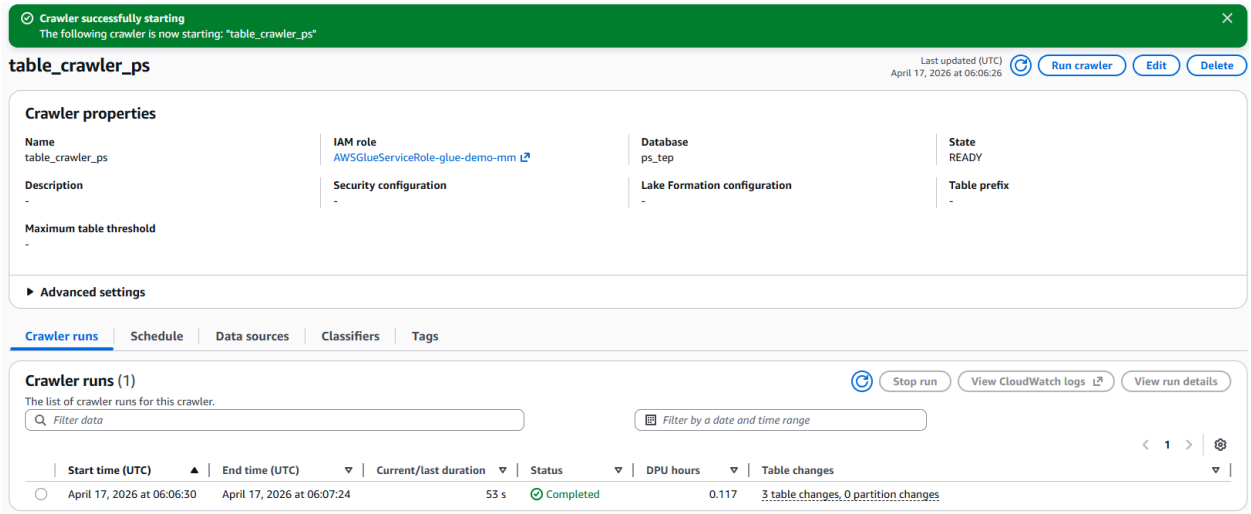




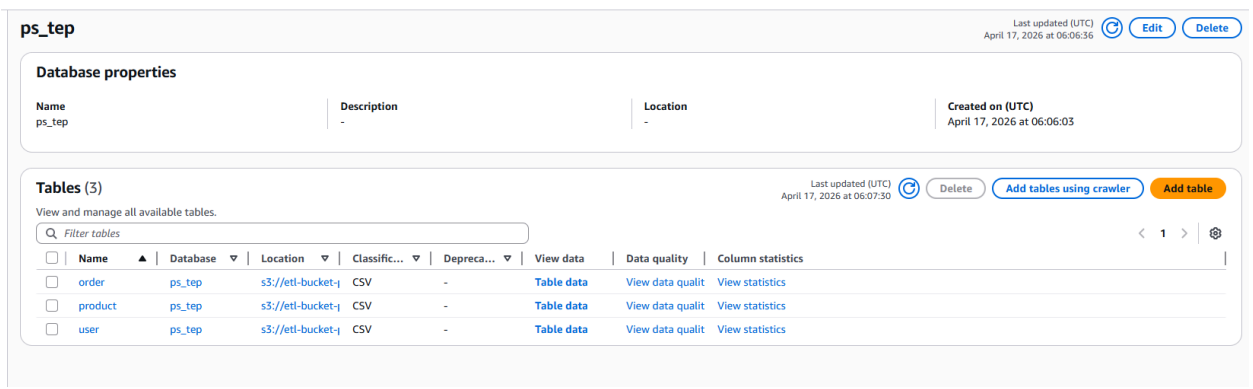




- Created AWS Glue Crawlers to scan the data



- Generated tables in Glue Data Catalog



6.2 Data Transformation

Performed using AWS Glue PySpark script: -

Data Cleaning

- Removed null values from key fields

```
# CLEAN DATA

orders_df = orders_df.dropna(subset=["UserID", "OrderID"])
```

Data Formatting

- Converted date fields into standard format

```
# FORMAT DATES

users_df = users_df.withColumn(    "SignupDate", to_date(trim(col("SignupDate")), "yyyy-MM-dd")

)

orders_df = orders_df.withColumn( "OrderDate",to_date(trim(col("OrderDate")), "yyyy-MM-dd")

)
```

Data Joining

- *Joined user and order datasets using UserID*

```
# JOIN DATA

df = orders_df.join(users_df, "UserID", "inner")
```

Derived Fields

- Created `AccountAge` using date difference

```
# ADD NEW COLUMN

df = df.withColumn(

   "AccountAge",

   datediff(col("OrderDate"), col("SignupDate")) / 365

)
```

Filtering

- Filtered records based on business logic

```
# FILTER DATA

df = df.filter(col("Amount") >= 50)
```

job_etl_ps

Script | Job details | Runs | Data quality | Schedules | Version Control

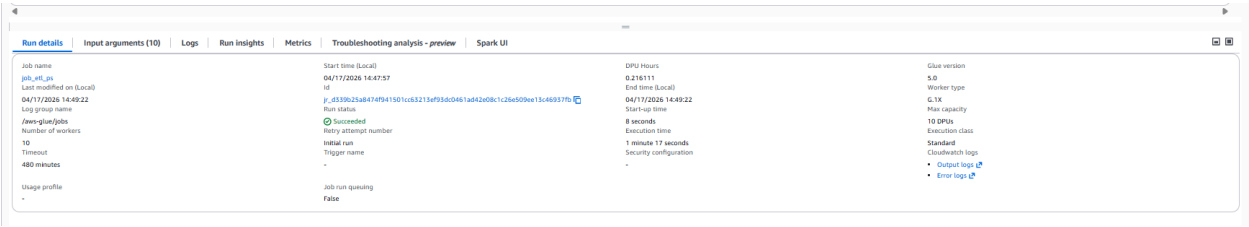
Script Info

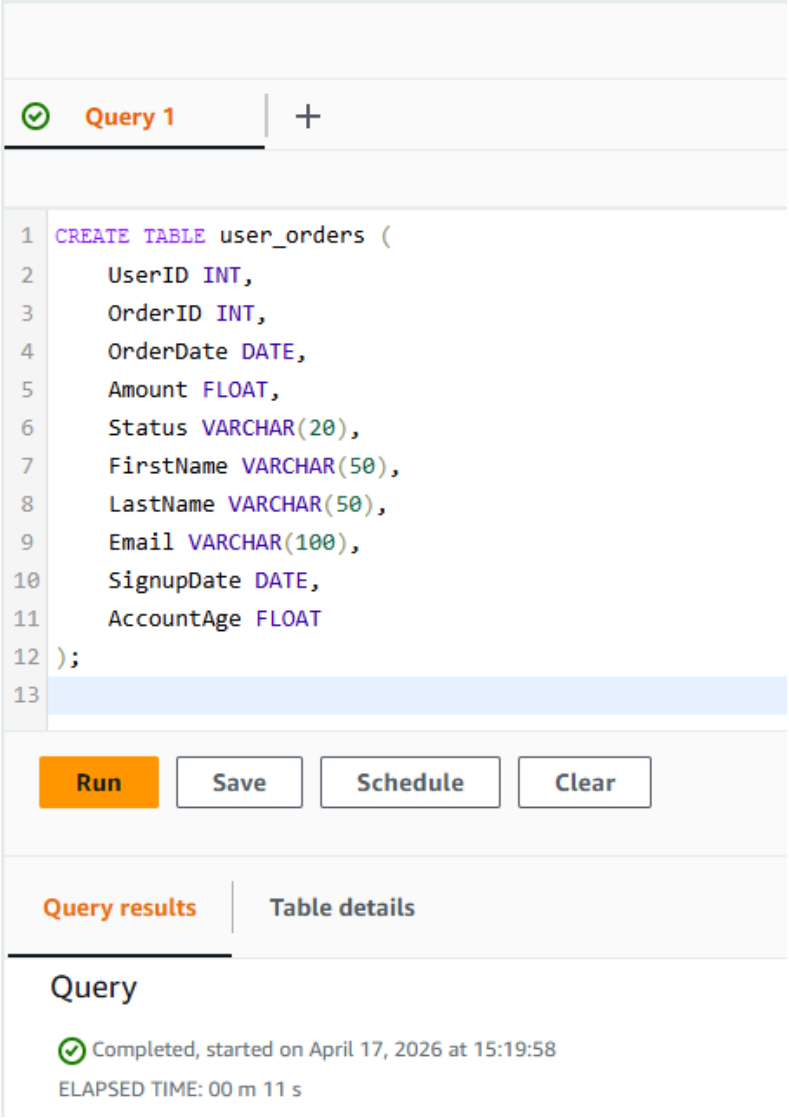
```python
import sys
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from pyspark.sql.functions import *

# Initialize Glue Job
args = getResolvedOptions(sys.argv, ['JOB_NAME'])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# LOAD DATA
users_df = glueContext.create_dynamic_frame.from_catalog(
    database="ps_tep", table_name="user").toDF()

orders_df = glueContext.create_dynamic_frame.from_catalog(
    database="ps_tep", table_name="order").toDF()

# CLEAN DATA
orders_df = orders_df.dropna(subset=["UserID", "OrderID"])

# FORMAT DATES
users_df = users_df.withColumn("SignupDate", to_date("SignupDate", "yyyy-MM-dd"))
orders_df = orders_df.withColumn("OrderDate", to_date("OrderDate", "yyyy-MM-dd"))

# JOIN DATA
df = orders_df.join(users_df, "UserID", "inner")

# ADD NEW COLUMN
df = df.withColumn(
    "AccountAge",
    datediff(col("OrderDate"), col("SignupDate")) / 365
)

# FILTER DATA
df = df.filter(col("Amount") > 50)

# LOAD TO S3 (PARQUET)
df.write \
  .mode("overwrite") \
  .parquet("s3://etl-bucket-ps/clean/")

# Commit Job
job.commit()
```

6.3 Data Loading

Load to Redshift

- Used COPY command to load data from S3
- Used IAM role for secure access





7. Redshift Table Schema

```
CREATE TABLE user_orders (
    UserID BIGINT,
    OrderID BIGINT,
    OrderDate DATE,
    Amount DOUBLE PRECISION,
    Status VARCHAR(20),
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100),
    SignupDate DATE,
    AccountAge DOUBLE PRECISION
);
```

8. Challenges Faced and Solutions

Issue	Solution
GlueContext not defined	Added initialization code
Table not found	Corrected database name
VPC / STS error	Used COPY command instead
Schema mismatch	Updated Redshift table data types
Date parsing error	Used trim() before conversion
Empty output	Fixed filter condition

10. Conclusion

This project successfully demonstrates building a scalable ETL pipeline using AWS Glue and Redshift. The use of Parquet format and COPY command improved performance and efficiency. The pipeline can be extended for real-time analytics and large-scale data processing.